IT112 Web System Development

Phase 4 – Form Submission and Data Insertion

Lab Report by TechNet

Group Members:

Mico Malana

Christian Silo

Jay Lloyd Chicogo

Nelson Andes

Allen Benedict

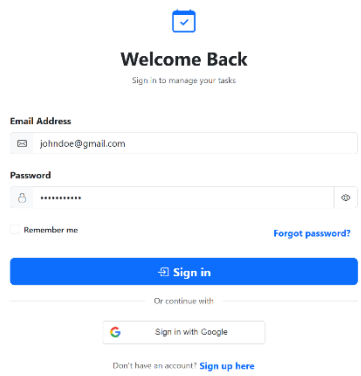
Instructor:

Guillermo V. Red, Jr., DIT

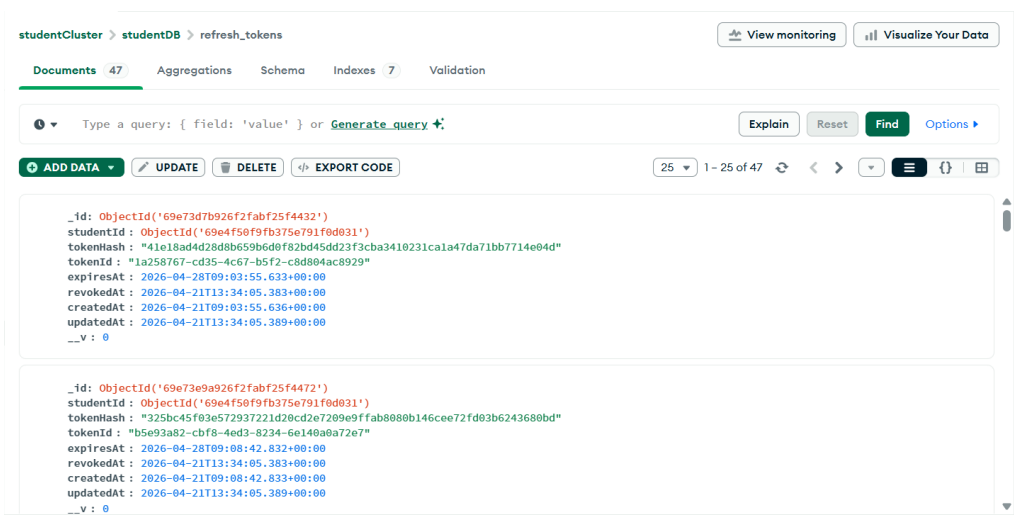
I. Form Interface

The screenshots below present the system's form interfaces and the corresponding documents stored in MongoDB, showing how user inputs are submitted and processed for various operations.

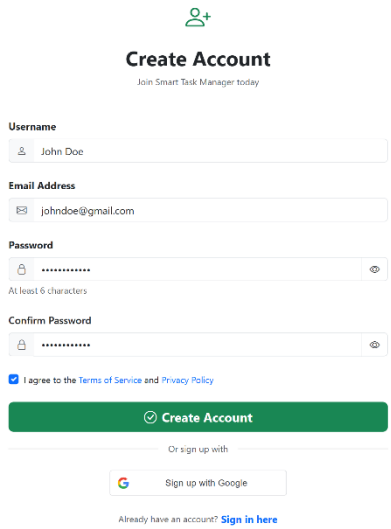
a. Login Form



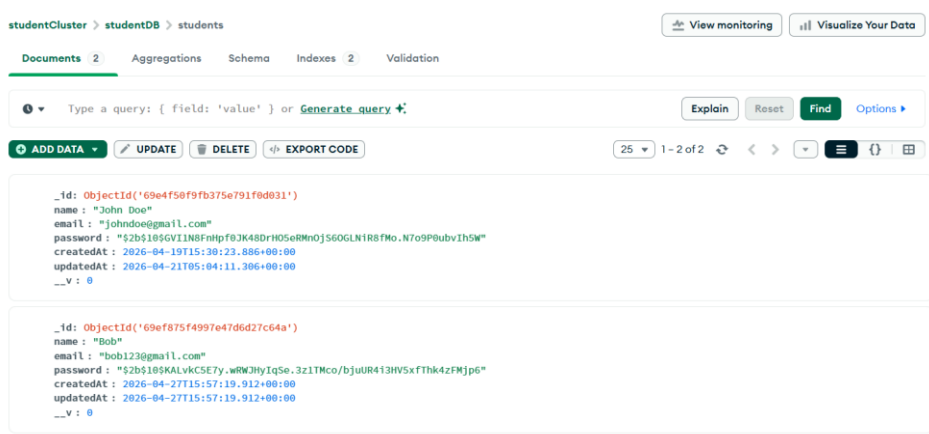
When a user logs in successfully, an access token and a refresh token are generated. The refresh token is stored in the refresh_tokens collection, while the access token is returned to the client for authenticated requests



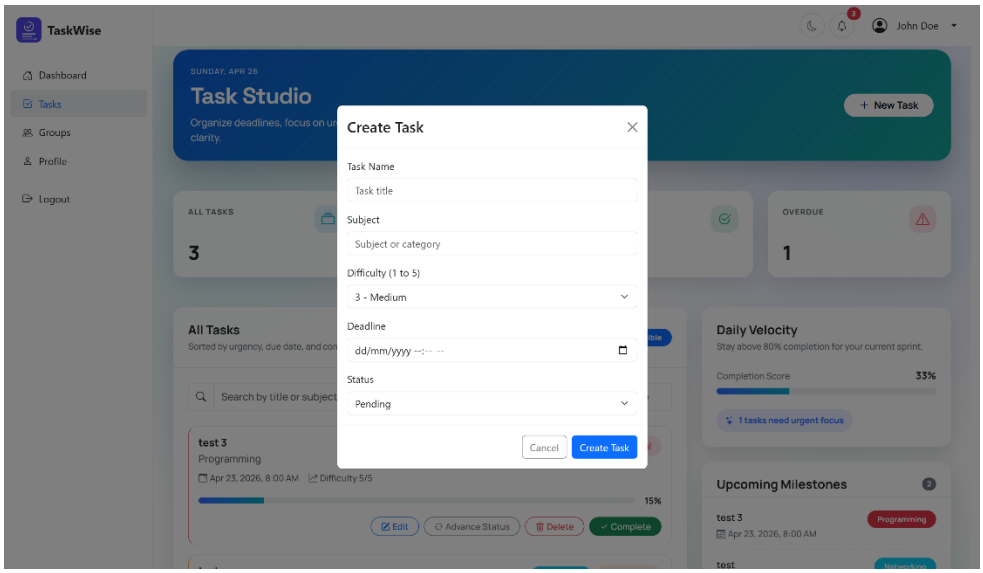
b. Register Form



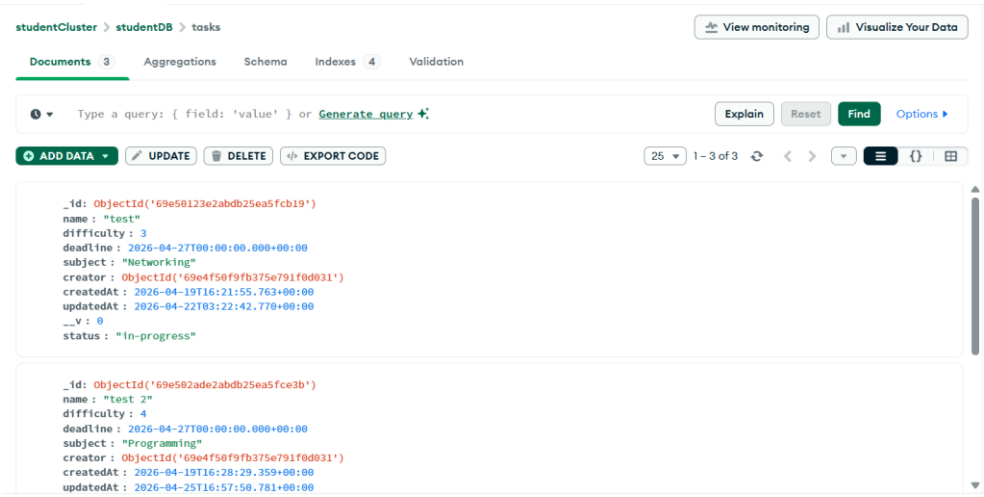
When a user successfully registers for an account, the provided information is validated and then stored in the students collection.



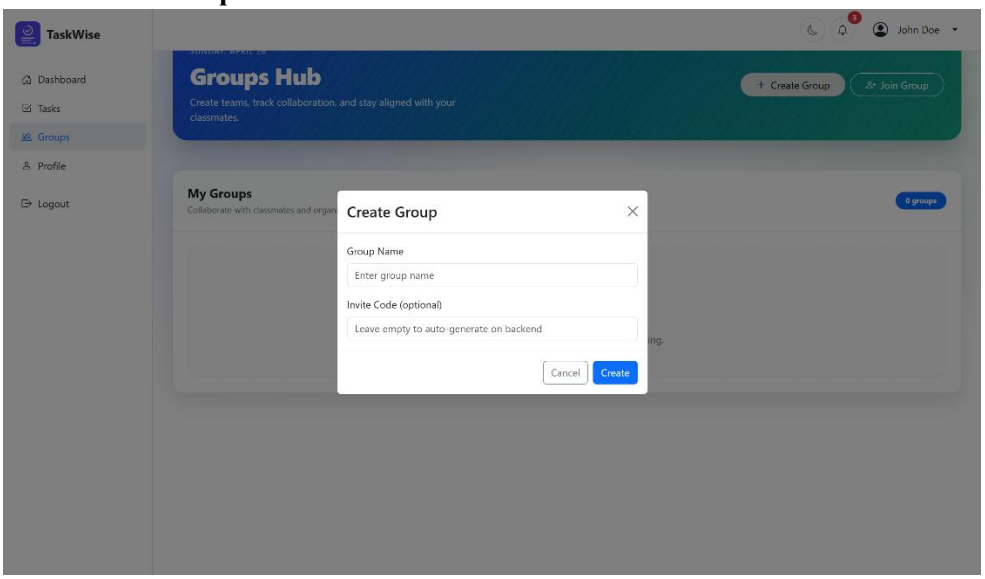
c. Create Task Form



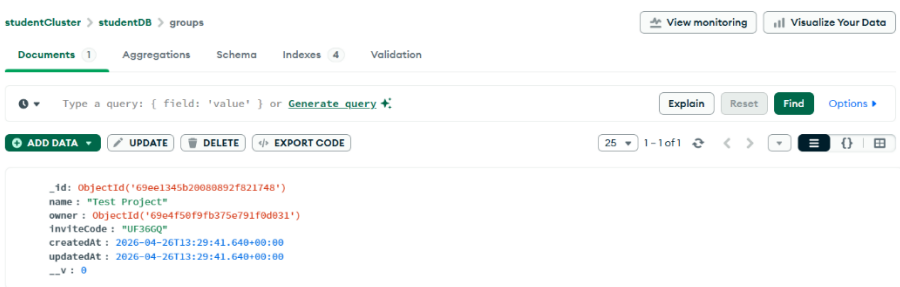
When a user successfully creates a task, the submitted data is validated and stored in the tasks collection. The task is associated with the authenticated user and can be viewed and managed within the system.



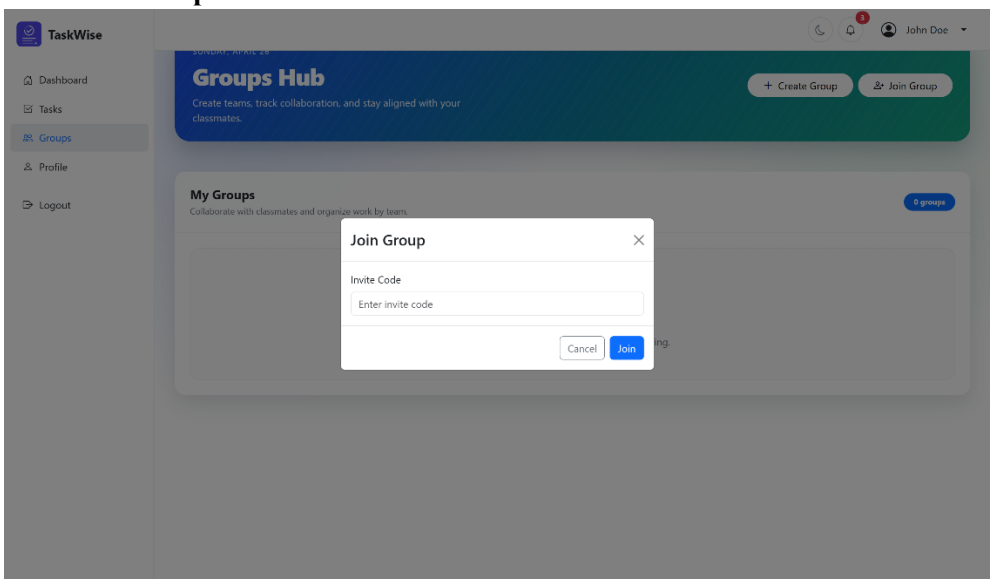
d. Create Group Form



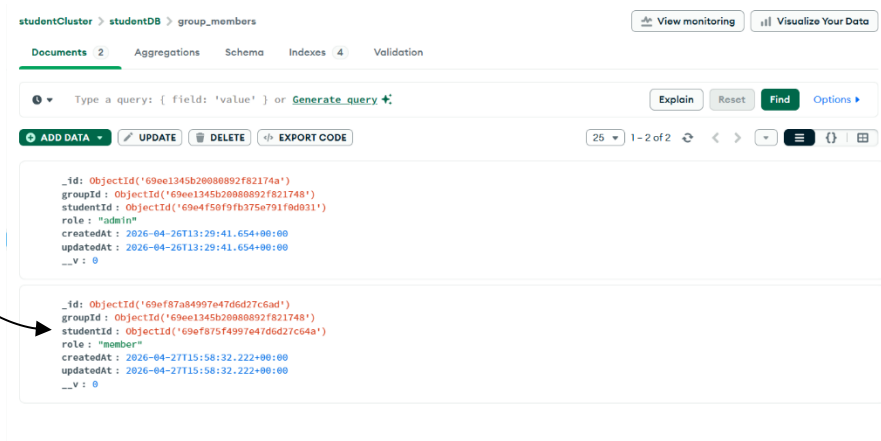
When a user successfully creates a group, the submitted data is validated and stored in the groups collection. The group is associated with the authenticated user and can be managed within the system.



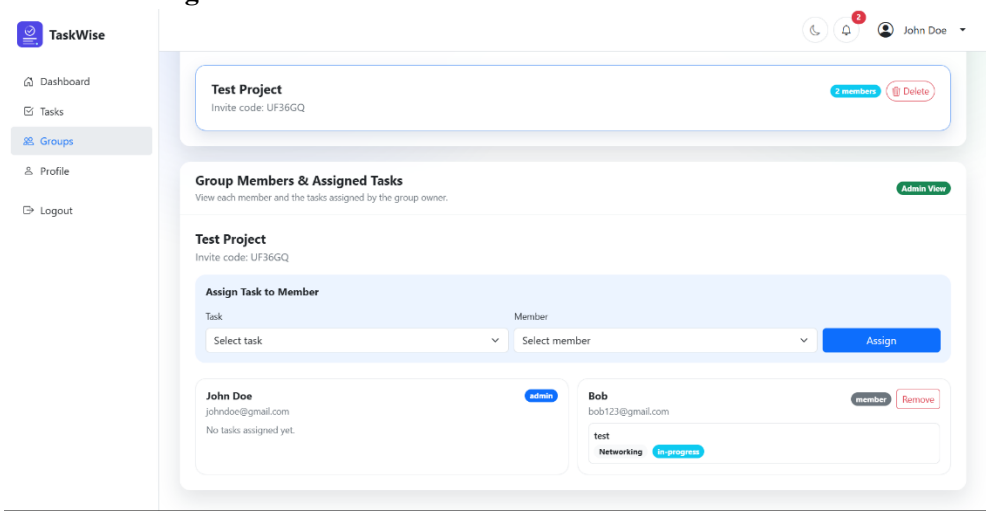
e. Join Group Form



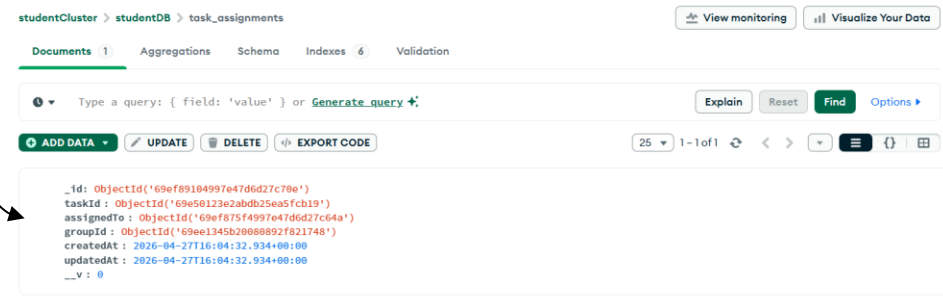
When a user submits a valid invite code, the system checks it and adds the user's ID to the group_members collection with the group they joined.



f. Task Assignment Form



When a user successfully assigns a task to a group member, the member's ID is stored in the task_assignments collection along with the corresponding task ID.



II. API Connection

The snippets below illustrate the implementation of API connections used to process user requests and interact with the backend.

a. Auth API Service

```ts
import axios from "axios";

const resolveApiBaseUrl = () : any => {
  const configuredBaseUrl = import.meta.env.VITE_API_URL;

  if (configuredBaseUrl) {
    return configuredBaseUrl.replace(/\/+$/, "");
  }

  return "";
};

const apiClient = axios.create({
  baseURL: resolveApiBaseUrl(),
  timeout: 15000,
  headers: {
    "Content-Type": "application/json",
  },
  withCredentials: true,
});

let refreshPromise = null;

export const setAuthToken = () : void => {};
export const setRefreshToken = () : void => {};

export const getAuthToken = () : null => {
  return null;
};

export const getRefreshToken = () : null => {
  return null;
};

export const clearAuthToken = () : void => {
  return;
};

const storeAuthSession = () : void => {};

const clearAuthSession = () : void => {
  clearAuthToken();
};

const requestTokenRefresh = async () : Promise<AxiosResponse<any, … => {
  const response = await axios.post(
    `${resolveApiBaseUrl()}/api/student/refresh`,
    {},
    {
      timeout: 15000,
      headers: { "Content-Type": "application/json" },
      withCredentials: true,
    },
  );

  return response;
};
```

```ts
apiClient.interceptors.response.use(
  (response : AxiosResponse<any, any, {}>) : AxiosResponse<any, any, {}> => response,
  async (error : any) : Promise<AxiosResponse<any, … => {
    const originalRequest = error?.config;
    const status = error?.response?.status;

    if (!originalRequest || status !== 401 || originalRequest._retry) {
      return Promise.reject(error);
    }

    if (
      typeof originalRequest.url === "string" &&
      (originalRequest.url.includes("/api/student/login") ||
        originalRequest.url.includes("/api/student/google") ||
        originalRequest.url.includes("/api/student/register") ||
        originalRequest.url.includes("/api/student/refresh"))
    ) {
      return Promise.reject(error);
    }

    originalRequest._retry = true;

    try {
      if (!refreshPromise) {
        refreshPromise = requestTokenRefresh().finally(() : void => {
          refreshPromise = null;
        });
      }

      await refreshPromise;

      return apiClient(originalRequest);
    } catch (refreshError) {
      clearAuthSession();
      return Promise.reject(refreshError);
    }
  },
);

export const extractAuthErrorMessage = (error : any) : string => {
  if (axios.isAxiosError(error)) {
    const messageFromServer =
      error.response?.data?.message || error.response?.data?.error;

    if (error.response?.status >= 500) {
      return "Server error while processing your request. Please try again in a moment.";
    }

    if (typeof messageFromServer === "string" && messageFromServer.trim()) {
      if (messageFromServer.toLowerCase().includes(searchString: "is not defined")) {
        return "Server error while processing your request. Please try again in a moment.";
      }

      return messageFromServer;
    }

    if (error.code === "ECONNABORTED") {
      return "Request timed out. Please try again.";
    }

    if (!error.response) {
      return "Unable to reach the server. Check your connection and API URL.";
    }
  }

  return "Request failed. Please verify your details and try again.";
};

export const authLogStudent = async (credentials : any) : Promise<AxiosResponse<any, … => {
  const payload = {
    email: credentials?.email?.trim() || "",
    password: credentials?.password || "",
  };

  const response = await apiClient.post(url: "/api/student/login", payload);
  storeAuthSession();

  return response;
};

export const authLogStudentWithGoogle = async (idToken : any) : Promise<AxiosResponse<any, … => {
  const payload = {
    idToken: typeof idToken === "string" ? idToken : "",
  };

  const response = await apiClient.post(url: "/api/student/google", payload);
  storeAuthSession();

  return response;
};

export const authLogoutStudent = async () : Promise<void> => {
  try {
    await apiClient.post(url: "/api/student/logout", {});
  } finally {
    clearAuthSession();
  }
};

export const authRegStudent = async (credentials : any) : Promise<AxiosResponse<any, … => {
  const payload = {
    name: credentials?.username?.trim() || "",
    email: credentials?.email?.trim() || "",
    password: credentials?.password || "",
  };

  return apiClient.post(url: "/api/student/register", payload);
};

export const getAuthStudent = async () : Promise<AxiosResponse<any, … => {
  return apiClient.get(url: "/api/student");
};

export { apiClient };
```

b. Dashboard API Service

```python
import { apiClient } from "../../auth/api/authApi";

export const updateTaskStatus = async (taskId : any, status : any) : Promise<AxiosResponse<any, … => {
  return apiClient.patch(url: `/api/task/update/${taskId}`, { status });
};

export const fetchDashboardData = async () : Promise<{ student: any; tas… => {
  const [studentResult, tasksResult] = await Promise.allSettled([
    apiClient.get(url: "/api/student"),
    apiClient.get(url: "/api/task"),
  ]);

  if (tasksResult.status === "rejected") {
    throw tasksResult.reason;
  }

  const student =
    studentResult.status === "fulfilled"
      ? studentResult.value?.data?.student ?? null
      : null;

  const tasks =
    tasksResult.value?.data?.tasks && Array.isArray(tasksResult.value.data.tasks)
      ? tasksResult.value.data.tasks
      : [];

  return {
    student,
    tasks,
  };
};
```

c. Tasks API Service

```python
import { apiClient } from "../../auth/api/authApi";

export const fetchTasks = async () : Promise<any> => {
  const response = await apiClient.get(url: "/api/task");
  const tasks = Array.isArray(response?.data?.tasks) ? response.data.tasks : [];
  return tasks;
};

export const createTask = async (taskPayload : any) : Promise<any> => {
  const response = await apiClient.post(url: "/api/task/create", taskPayload);
  return response?.data?.newTask;
};

export const updateTask = async (taskId : any, taskPayload : any) : Promise<any> => {
  const response = await apiClient.patch(url: `/api/task/update/${taskId}`, taskPayload);
  return response?.data?.newTask;
};

export const deleteTask = async (taskId : any) : Promise<void> => {
  await apiClient.delete(url: `/api/task/delete/${taskId}`);
};
```

d. Groups API Service

```python
import { apiClient } from "../../auth/api/authApi";

export const fetchGroups = async () : Promise<any> => {
  const response = await apiClient.get(url: "/api/group");
  const groups = Array.isArray(response?.data?.groups) ? response.data.groups : [];
  return groups;
};

export const createGroup = async (payload : any) : Promise<any> => {
  const response = await apiClient.post(url: "/api/group/create", payload);
  return response?.data?.newGroup ?? response?.data?.group ?? null;
};

export const joinGroup = async (payload : any) : Promise<any> => {
  const response = await apiClient.post(url: "/api/group/join", payload);
  return response?.data?.member ?? response?.data?.group ?? null;
};

export const removeGroup = async (groupId : any) : Promise<void> => {
  await apiClient.delete(url: `/api/group/delete/${groupId}`);
};

export const removeGroupMember = async (groupId : any, memberId : any) : Promise<void> => {
  await apiClient.delete(url: `/api/group/${groupId}/member/${memberId}`);
};

export const leaveGroup = async (groupId : any) : Promise<void> => {
  await apiClient.delete(url: `/api/group/${groupId}/leave`);
};

export const fetchGroupDetails = async (groupId : any) : Promise<{ group: any; membe… => {
  const response = await apiClient.get(url: `/api/group/${groupId}/details`);

  return {
    group: response?.data?.group ?? null,
    members: Array.isArray(response?.data?.members) ? response.data.members : [],
  };
};

export const fetchAssignableTasks = async () : Promise<any> => {
  const response = await apiClient.get(url: "/api/task");
  return Array.isArray(response?.data?.tasks) ? response.data.tasks : [];
};

export const assignTaskToMember = async ({ taskId, assignedTo, groupId } : { taskId: any; assignedTo: …) : Promise<any> => {
  const response = await apiClient.post(url: "/api/task/assign", {
    taskId,
    assignedTo,
    groupId,
  });

  return response?.data?.assigned ?? null;
};
```

III. GitHub Commits

feat: add useProfileData hook for profile state and API handling (#41)
jayl-ch and Sannie-dev authored 34 minutes ago — Verified — a6a910f

Add tasks API service and useTasksData hook (#40)

```
* feat: add tasks API service functions

* feat: add useTasksData hook to consume tasks API

---------

Co-authored-by: Christian <christiansanniesaelsilo@gmail.com>
```

jayl-ch and Sannie-dev authored 48 minutes ago — Verified — 3886bfb

Add groups API service and useGroupsData hook (#39)

```
* feat: add groups API service functions

* feat: add useGroupsData hook to consume groups API

---------

Co-authored-by: Christian <christiansanniesaelsilo@gmail.com>
```

jayl-ch and Sannie-dev authored 1 hour ago — Verified — e34854d

Feature/frontend auth pages (#31)
Sannie-dev authored 4 days ago — Verified — 7de47a4

Feature/dashboard page (#33)
Sannie-dev authored 2 days ago — Verified — 7dd9bad

IV. Short Reflections

a. Mico Malana (Project Manager, Documentation Officer)

During this Phase, I had difficulty monitoring our progress and documenting the system because of issues in the frontend. We needed to fix it first before connecting with backend, which also delayed our documentation. This experience helped me improve in managing priorities and keeping the documentation aligned with the actual system progress.

b. Christian Sannie Silo (Frontend Developer)

As the Frontend Developer, I was responsible for creating the API service functions and the hooks for consuming them in the application. I handled data fetching and made sure the frontend and backend are connected properly, although sometimes I encounter errors when handling API responses. This experience helped me improve my understanding in API integration and organizing frontend code.

c. Jay Lloyd Chicogo (Backend Developer, GitHub Manager)

As the GitHub Manager and Backend Developer, I was responsible for managing the repository and assisting in connecting the frontend to the backend. I ensured that branches, commits, and pull requests are organized properly while helping integrate the API into the frontend. This experience helped me improve my skills in Git and in supporting smooth communication between frontend and backend.

d. John Nelson Andes (Database Manager)

Working on this task helped me understand how the frontend connects to the backend. I was able to send form data and see it saved in MongoDB, which made the process feel real and complete. I also took responsibility for checking if the inputs were correctly stored in the database. It showed me how important it is to test and verify data, not just rely on alerts. Overall, it was a simple but meaningful experience in building a full feature.

e. Allen Benedict Asuncion (Tester & Debugger)

As the Tester and Debugger, I was responsible for checking if the form interfaces respond correctly and work as expected. I tested different inputs to make sure the system behaves properly and returns the correct results. This helped ensure that the features are functioning well before deployment and improved my attention to detail in identifying issues.